

CAPÍTULO 04 — GUÍA DEL ESTUDIANTE

Spring MVC real aplicado a tu proyecto: endpoints, DTOs, parámetros, códigos HTTP y validación

Programación Aplicada 2026-2

Tu misión

Observarás al profesor construir la API de Cliente en ParkFlow 360. Tú NO debes copiar Cliente. Debes aplicar exactamente el mismo patrón sobre la entidad padre 1:N que seleccionaste de tu proyecto en los capítulos anteriores.

1. Qué debes tener antes de comenzar

- Proyecto Spring Boot del Capítulo 03 funcionando.
- Java 21 y Maven configurados.
- Tus dos entidades 1:N ya seleccionadas.
- Identificado cuál es la entidad PADRE y cuál la DEPENDIENTE.
- Package base de tu proyecto definido.

2. Regla de transferencia profesor → estudiante

Profesor — ParkFlow	Tú — Proyecto asignado
Proyecto: parkflow-backend	Proyecto: <tu-proyecto>-backend
Package: com.parkflow	Package: com.<tu-proyecto>
Entidad padre: Cliente	Tu entidad padre 1:N
Entidad hija: Vehiculo	Tu entidad dependiente 1:N
/api/clientes	/api/<tu-recurso-padre>
CrearClienteRequest	Crear<TuEntidad>Request
ClienteResponse	<TuEntidad>Response

Ejemplo

Si tu proyecto fuera BiblioSmart y tu relación fuera Socio 1:N Prestamo, el profesor usa Cliente pero tú podrías trabajar con Socio. Tus nombres deben provenir de tu dominio real.

3. Agrega Validation al pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

4. Crea la estructura de paquetes de tu entidad padre

```
com.<tu-proyecto>
```

```

└─ <modulo-padre>
    └─ domain
        └─ <EntidadPadre>.java
    └─ application
        └─ <EntidadPadre>Service.java
    └─ infrastructure
        └─ adapter
            └─ in
                └─ web
                    └─ <EntidadPadre>Controller.java
                        └─ dto
                            └─ Crear<EntidadPadre>Request.java
                                └─ <EntidadPadre>Response.java

```

No inventes nombres

Usa el vocabulario de tu proyecto y tu modelo de datos. Los packages van en minúsculas; las clases en PascalCase.

5. Diseña tu Request DTO

- Selecciona entre 3 y 5 atributos realmente necesarios para crear la entidad.
- No incluyas la PK si la generará el sistema.
- Define qué campos son obligatorios.
- Decide qué formatos deben validarse.
- No copies las validaciones de Cliente si no aplican a tu dominio.

```

public record Crear<TuEntidad>Request(
    @NotBlank(message = "...")
    String campoObligatorio,

    @Size(max = 100, message = "...")
    String otroCampo
) {}

```

6. Diseña tu Response DTO

```

public record <TuEntidad>Response(
    Long id,
    String campo1,
    String campo2
) {}

```

Debes poder defender

¿Por qué el Request y el Response son distintos? ¿Qué atributo podría entrar pero no salir, o salir pero no entrar?

7. Servicio temporal en memoria

```
@Service
public class <TuEntidad>Service {
    private final List<<TuEntidad>> elementos = new ArrayList<>();
    private final AtomicLong secuencia = new AtomicLong(0);

    public <TuEntidad> registrar(/* parámetros */) {
        // construir tu objeto con id generado
        // agregarlo a la lista
        // devolverlo
    }

    public List<<TuEntidad>> listar() {
        return List.copyOf(elementos);
    }

    public Optional<<TuEntidad>> buscarPorId(Long id) {
        return elementos.stream()
            .filter(e -> e.getId().equals(id))
            .findFirst();
    }
}
```

Importante

No intentes conectar PostgreSQL todavía. La colección se usa solo para aprender HTTP/Spring MVC. La persistencia real llegará en capítulos posteriores.

8. Crea tu Controller

```
@RestController
@RequestMapping("/api/<recurso-padre>")
public class <TuEntidad>Controller {

    private final <TuEntidad>Service service;

    public <TuEntidad>Controller(<TuEntidad>Service service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<<TuEntidad>Response> crear(
        @Valid @RequestBody Crear<TuEntidad>Request request) {
        // registrar
        // mapear a response
        // responder 201 CREATED
    }

    @GetMapping
    public List<<TuEntidad>Response> listar(
        @RequestParam(required = false) String filtro) {
        // listar y aplicar filtro si corresponde
    }

    @GetMapping("/{id}")
    public ResponseEntity<<TuEntidad>Response> buscarPorId(
        @PathVariable Long id) {
        // 200 si existe
    }
}
```

```

    // 404 si no existe
  }
}

```

9. Lo que debes entender de cada anotación

Anotación	Debes explicarla con tus palabras
@RestController	
@RequestMapping	
@PostMapping	
@GetMapping	
@RequestBody	
@Valid	
@PathVariable	
@RequestParam	

10. Pruebas mínimas obligatorias

Prueba	Resultado esperado
POST válido	201 Created + JSON del recurso
POST con campo obligatorio vacío	400 Bad Request
POST con formato inválido	400 Bad Request
GET lista	200 OK + arreglo JSON
GET por id existente	200 OK
GET por id inexistente	404 Not Found
GET con parámetro opcional	Filtra o modifica la consulta según tu diseño

11. Archivo HTTP sugerido

```

### Crear recurso válido
POST http://localhost:8080/api/<recurso-padre>
Content-Type: application/json

{
  "campo1": "valor",
  "campo2": "valor"
}

### Listar
GET http://localhost:8080/api/<recurso-padre>

### Buscar por id
GET http://localhost:8080/api/<recurso-padre>/1

### Buscar inexistente
GET http://localhost:8080/api/<recurso-padre>/999

```

12. Debes documentar tu contrato HTTP

Verbo	Ruta	Entrada	Salida	Status
-------	------	---------	--------	--------

POST	/api/...	JSON Request DTO	Response DTO	201 / 400
GET	/api/...	Query param opcional	Lista de DTOs	200
GET	/api/.../{id}	Path variable	Response DTO	200 / 404

13. No hagas estas cosas

- No copies Cliente/Vehiculo literalmente.
- No expongas tu clase de dominio como contrato definitivo de la API.
- No uses @RequestBody sin entender qué JSON espera.
- No pongas 200 para todo.
- No conectes JPA todavía.
- No agregues lógica de negocio compleja dentro del Controller.

14. Preguntas para tu defensa

1. ¿Por qué elegiste esa entidad como padre?
2. ¿Cuál es la URL base de tu recurso y por qué?
3. ¿Qué diferencia hay entre Request DTO y Response DTO?
4. ¿Qué validaciones elegiste y qué regla protege cada una?
5. ¿Qué diferencia hay entre @PathVariable y @RequestParam?
6. ¿Por qué tu POST responde 201?
7. ¿Qué ocurrirá con tus datos si reinicias la aplicación en este capítulo?
8. ¿Dónde está el adaptador de entrada de tu módulo?

15. Evidencia que debes entregar

- Captura o salida de POST válido.
- Evidencia de POST inválido con 400.
- Evidencia de GET por id existente.
- Evidencia de GET inexistente con 404.
- Tabla de contrato HTTP completada.
- Código en packages correctos.
- Commit Git.

```
git add .
git commit -m "feat: add REST API and validation for <tu-entidad>"
```

16. Definition of Done

Terminaste el capítulo si

Puedes construir, ejecutar y explicar un POST y dos GET de tu entidad padre; utilizas Request/Response DTO; validas la entrada; distingues PathVariable, RequestParam y RequestBody; y respondes con 201, 200, 400 y 404 cuando corresponda.

17. Puente al siguiente capítulo

Lo que viene

En el próximo capítulo conectaremos Spring Boot con PostgreSQL y comenzaremos a estudiar JPA/Hibernate. Entonces la lista en memoria desaparecerá y el backend deberá respetar el modelo físico que ya construyeron en la base de datos.